

GESTION D'UN SERVICE WEB

Apache

OBJECTIFS

- Installer Apache 2 et comprendre son fonctionnement
- Configurer SSH et se connecter via Putty et WinSCP
- Installer et configurer HTTPS et OPENSSL
- Créer un certificat auto-signé et une Autorité de Certification (AC)
- Importer le certificat de l'AC dans le navigateur client et tester l'accès HTTPS (sans alerte de sécurité)
- Rediriger automatiquement les requêtes http vers https
- Personnaliser le serveur Apache
- Sécuriser les répertoires du site

1. INSTALLER APACHE

Vérification d'usage et installation d'Apache (commande **sudo** ou **su** - si nécessaire) :

<code>apt update</code>	vérifie les dépôts et met à jour la liste des nouveaux paquets en attente
<code>apt upgrade</code>	recupère les mises à jour potentielles et les installe
<code>apt install apache2</code>	installation le service Apache 2

BLOC 2



Remarque : Vérifier la présence de **apache2** dans le répertoire **etc/init.d** qui stocke les scripts de lancement des daemons.

`ip a`

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:37:de:6b brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.252.129/24 brd 192.168.252.255 scope global dynamic ens33
        valid_lft 1789sec preferred_lft 1789sec
    inet6 fe80::20c:29ff:fe37:de6b/64 scope link
        valid_lft forever preferred_lft forever
```

`systemctl status apache2`

```
• apache2.service - The Apache HTTP Server
   Loaded: loaded (/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Tue 2023-09-19 18:41:58 CEST; 6min ago
     Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 1439 (apache2)
      Tasks: 55 (limit: 2265)
     Memory: 9.2M
        CPU: 103ms
    CGroup: /system.slice/apache2.service
            └─1439 /usr/sbin/apache2 -k start
              └─1441 /usr/sbin/apache2 -k start
                └─1442 /usr/sbin/apache2 -k start

sept. 19 18:41:58 srvtdeb systemd[1]: Starting apache2.service - The Apache HTTP Server...
sept. 19 18:41:58 srvtdeb apache2ctl[1438]: AH00558: apache2: Could not reliably determine the server's
sept. 19 18:41:58 srvtdeb systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-16/16 (END)
```

APACHE HTTPD



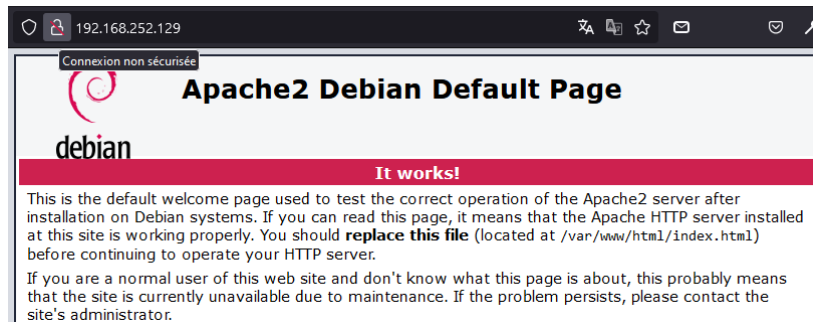
BTS SIO – SISR



Pour connaître la version d'Apache installée sur le serveur :

```
apache2 -v
```

Vérifier que le serveur est joignable :



Commandes et dossiers à connaître :

```
systemctl status apache2
systemctl restart apache2
systemctl stop apache2
systemctl reload apache2
```

Afficher le statut du serveur web Apache
Redémarrer le serveur web Apache
Stopper le serveur web Apache
Recharger le service web Apache

Les fichiers de base de notre serveur web Apache se trouvent ici :

```
cd /etc/apache2
ls -la
```

2 dossiers sont importants :

- Sites-available
- Sites-enabled

```
784381 drwxr-xr-x  8 root root  4096 19 sept. 18:41 .
783361 drwxr-xr-x 66 root root  4096 19 sept. 18:56 ..
784397 -rw-r--r--   1 root root  7178 13 avril 05:26 apache2.conf
784398 drwxr-xr-x  2 root root  4096 19 sept. 18:41 conf-available
784404 drwxr-xr-x  2 root root  4096 19 sept. 18:41 conf-enabled
784405 -rw-r--r--   1 root root  1782  9 juin  2022 envvars
784406 -rw-r--r--   1 root root 31063  9 juin  2022 magic
784407 drwxr-xr-x  2 root root 12288 19 sept. 18:41 mods-available
784549 drwxr-xr-x  2 root root  4096 19 sept. 18:41 mods-enabled
784550 -rw-r--r--   1 root root   274 13 avril 05:14 ports.conf
784551 drwxr-xr-x  2 root root  4096 19 sept. 18:41 sites-available
784554 drwxr-xr-x  2 root root  4096 19 sept. 18:41 sites-enabled
```

Le dossier **Sites-available** contient les sites DISPONIBLES sur le serveur web Apache

Le dossier **Sites-enabled** contient les sites ACTIFS sur le serveur web Apache

Les fichiers de log (accès au serveur web Apache et erreurs de log) sont le répertoire **/var/log/apache2/**

```
1046480 drwxr-x--- 2 root adm 4096 19 sept. 18:41 .
1046481 drwxr-xr-x 7 root root 4096 19 sept. 18:41 ..
1046688 -rw-r----- 1 root adm 555 19 sept. 18:43 access.log
1046687 -rw-r----- 1 root adm 864 19 sept. 18:51 error.log
1046689 -rw-r----- 1 root adm 0 19 sept. 18:41 other_vhosts_access.log
```

Le fichier **access.log** affiche les connexions au serveur web Apache.

Le fichier **error.log** affiche les erreurs de connexion au serveur web Apache.

2. CONFIGURER L'ACCES SSH SUR LE SERVEUR DEBIAN

Depuis un client (Windows ou Linux) il est impossible de se connecter au serveur Apache :

```
PS C:\Users\Utilisateur> ssh 192.168.252.129
ssh: connect to host 192.168.252.129 port 22: Connection refused
```

Pour y accéder à distance, il faut installer OpenSSH-server puis configurer l'accès SSH :

```
apt install openssh-server
```

```
PS C:\Users\Utilisateur> ssh 192.168.252.129
The authenticity of host '192.168.252.129 (192.168.252.129)' can't be established.
ECDSA key fingerprint is SHA256:gh8l8d6qejt61IpVB/+7oIggqDpqCnpGXyTfn9+gDs.
```

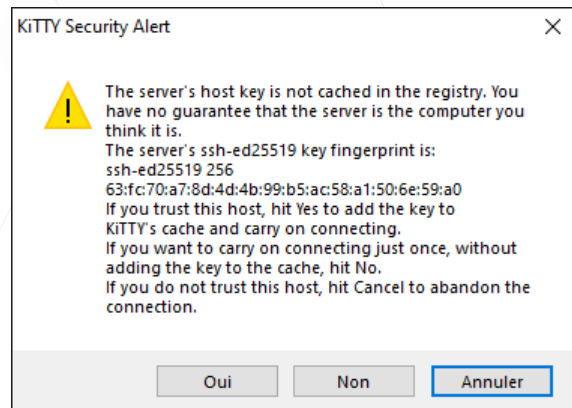
Ou avec Putty :

Specify the destination you want to connect to

Host Name (or IP address)	Port
192.168.252.129	22

Connection type:

☐ Raw ☐ Telnet ☐ Rlogin ☒ SSH ☐ ADB



```
login as: etudinfo
etudinfo@192.168.252.129's password:
Linux srvdeb 6.1.0-10-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.38-1 (2023-07-14) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Sep 19 21:16:44 2023 from 192.168.252.1
etudinfo@srvdeb:~$
```

Remarque : en ligne de commande il faut préciser l'utilisateur distant avec lequel vous souhaitez vous connecter sinon par défaut il tentera une connexion avec l'utilisateur de votre session locale

```
ssh utilisateurdistant@ip-serveur
ssh etudinfo@192.168.252.129
```

```
PS C:\Users\Utilisateur> ssh etudinfo@192.168.252.129
etudinfo@192.168.252.129's password:
Linux srvdeb 6.1.0-10-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.38-1 (2023-07-14) x86_64
```



Pour des raisons de sécurité, l'accès SSH en tant que root n'est pas possible par défaut mais il est possible d'activer l'accès root en SSH de la manière suivante :

- Déplacez-vous dans le dossier **/etc/ssh**
- Ouvrez le fichier **sshd_config**
- Modifiez la section **# Authentication** ainsi :
 - Remplacer **prohibit-password** par **yes**
 - Enlever le # devant **PermitRootLogin** et **StrictModes**
- Sauvegardez les modifications
- Relancez le service SSH sur votre machine Debian et testez l'accès

```
# Authentication:
#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
```

Cela reste néanmoins fortement déconseillé d'utiliser un accès en tant que Root et de laisser l'accès root en mode SSH.

3. CONFIGURATION DE L'ACCES HTTP DU SERVEUR WEB APACHE

Pour le moment, après l'installation d'Apache, seul l'accès via le port HTTP (80) est actif par défaut. Pour le vérifier nous pouvons effectuer les vérifications suivantes :

```
cat /etc/apache2/sites-enabled/000-default.conf
```

On remarque que le site **000-default.conf** est bien actif sur le serveur web Apache puisqu'il fait partie du répertoire **sites-enabled** avec la commande **ls /etc/apache2/sites-enabled**

```
<VirtualHost *:80>
ServerAdmin webmaster@localhost
DocumentRoot /var/www/html

ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined

</VirtualHost>
```

Emplacement des
fichiers de log



Pour accéder à la page web (en http), nous avons saisi précédemment l'adresse IP du serveur web Apache. Nous allons modifier le fichier **000-default.conf** de manière à afficher la page en saisissant l'IP puis un autre test avec le nom de domaine local.

Modifier le fichier avec la commande :

```
nano /etc/apache2/sites-enabled/000-default.conf
```

Le fichier **virtualhost** du site web par défaut d'Apache s'ouvre et ajouter la ligne :

```
# Le nom de votre serveur avec son nom de domaine local (le FQDN de votre serveur)
ServerName srvdeb.sisr.tp
```

Ainsi, nous pourrions accéder directement à la page par défaut d'Apache en saisissant le nom de domaine dans la barre d'adresse du navigateur.

```
<VirtualHost *:80>
# The ServerName directive sets the request scheme, hostname and port that
# the server uses to identify itself. This is used when creating
# redirection URLs. In the context of virtual hosts, the ServerName
# specifies what hostname must appear in the request's Host: header to
# match this virtual host. For the default virtual host (this file) this
# value is not decisive as it is used as a last resort host regardless.
# However, you must set it for any further virtual host explicitly.
#ServerName www.example.com

ServerAdmin webmaster@localhost
ServerName srvdeb.sisr.tp
DocumentRoot /var/www/html

# Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
# error, crit, alert, emerg.
# It is also possible to configure the loglevel for particular
# modules, e.g.
#LogLevel info ssl:warn

ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined

# For most configuration files from conf-available/, which are
# enabled or disabled at a global level, it is possible to
# include a line for only one particular virtual host. For example the
# following line enables the CGI configuration for this host only
# after it has been globally disabled with "a2disconf".
#Include conf-available/serve-cgi-bin.conf
</VirtualHost>
```

Enregistrer le fichier et relancer le service Apache.

4. CONFIGURATION DE L'ACCES HTTPS DU SERVEUR WEB APACHE

Apache propose un accès HTTPS mais *cet accès n'est pas activé par défaut*. Le site web HTTPS par défaut se trouve dans **/etc/apache2/sites-available**, sous le nom **default-ssl.conf** :

Nous pouvons activer le site web HTTPS **default-ssl.conf** de la manière suivante :

```
root@srvdeb:~# ls -la /etc/apache2/sites-available/
total 20
784551 drwxr-xr-x 2 root root 4096 19 sept. 18:41 .
784381 drwxr-xr-x 8 root root 4096 19 sept. 18:41 ..
784552 -rw-r--r-- 1 root root 1316 20 sept. 19:01 000-default.conf
784553 -rw-r--r-- 1 root root 6195 13 avril 05:24 default-ssl.conf
```

- Relancez Apache pour activer le site HTTPS par défaut :

```
a2ensite default-ssl.conf
```

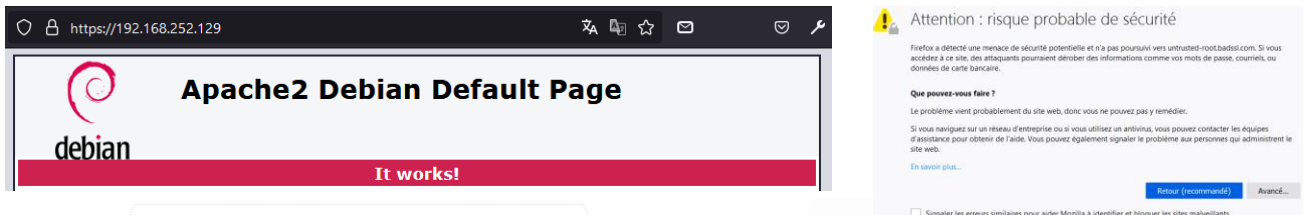
- Activez le mode SSL sur le serveur web Apache :

```
systemctl reload apache2
```

- Redémarrez Apache pour valider l'activation du mode SSL :

```
a2enmod ssl
```

- Testez l'accès à votre serveur web en mode HTTPS en saisissant **https://IP-votre-serveur-Debian** :



Remarque : L'accès HTTPS est fonctionnel mais le navigateur affiche une alerte liée au certificat auto-signé et l'absence d'Autorité de Certification, cela est indiqué par le navigateur puisque nous sommes en présence d'un **certificat auto-signé** bouton **Avancé...**

Par défaut, Apache a généré un certificat autosigné valable 10 ans mais ce certificat n'a pas été validé par une Autorité de Certification reconnue par le navigateur.

Analyse et configuration du fichier **default-ssl.conf** (attention, ici, nous pourrions copier/coller le fichier d'origine pour préserver l'original et créer ucd /etc/n fichier personnalisé pour notre nouveau site https) :

```
copy default-ssl.conf default-ssl.conf.sav
nano /etc/apache2/sites-available/default-ssl.conf
```

Pour afficher dans la liste des sites disponibles et éditer le contenu.

Ainsi, nous pourrions accéder directement à la page HTTPS par défaut d'Apache en saisissant le nom de domaine dans la barre d'adresse du navigateur.

```
<VirtualHost *:443>
    ServerAdmin webmaster@localhost

    DocumentRoot /var/www/html

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf

    #
    # SSL Engine Switch:
    # Enable/Disable SSL for this virtual host.
    SSLEngine on

    #
    # A self-signed (snakeoil) certificate can be created by installing
    # the ssl-cert package. See
    # /usr/share/doc/apache2/README.Debian.gz for more info.
    # If both key and certificate are stored in the same file, only the
    # SSLCertificateFile directive is needed.
    SSLCertificateFile      /etc/ssl/certs/ssl-cert-snakeoil.pem
    SSLCertificateKeyFile    /etc/ssl/private/ssl-cert-snakeoil.key
```

Emplacement des
fichiers de log

Emplacement des
fichiers de
certificat auto-signé du

Ajouter le nom de domaine local qui correspond à notre FQDN **ServerName srvdeb.sisr.tp**, à la suite de la ligne **ServerAdmin**. Relancez Apache :

```
systemctl restart apache2
```

L'accès HTTPS avec l'adresse **https://srvdeb.sisr.tp** est fonctionnel (avec l'alerte de sécurité).

5. CONFIGURATION OPENSSL

Dans cette partie, nous allons installer le module OPENSSL sur la machine Debian afin de créer une Autorité de Certification qui signera les certificats émis.



Installez OpenSSL sur votre machine Debian en saisissant :

```
apt install openssl
```

a) CREATION DE L'AUTORITE DE CERTIFICATION (CA)

Ici, nous n'aurons pas recours à une autorité reconnue mais à nous-mêmes (nous serons autorité de certification).

Nous allons créer les certificats SSL dans un dossier spécifique pour plus de clarté. Cette phase comporte 2 parties :

- ✓ la création de la **clé privée de l'Autorité de Certification** (fichier **.key**)
- ✓ la création du **certificat auto-signé de l'Autorité de Certification** (fichier **.crt**)

CREATION DE LA CLE PRIVEE DE L'AUTORITE DE CERTIFICATION

- Créez un dossier **ssl** dans **/etc/apache2** :

```
mkdir /etc/apache2/ssl
```

- Placez-vous dans le dossier ssl créé précédemment de manière à générer les certificats dans ce dossier
- Créez la clé privée de votre Autorité de Certification :

```
openssl genrsa 4096 > ca.key
```

Remarque : Ici, la clé est générée selon un algorithme de chiffrement asymétrique de type RSA 4096 bits et portera le nom de **ca.key**. Nous générons une clé privée sans "pass phrase" pour simplifier mais il faudrait, dans la pratique, protéger cette clé par un mot de passe fort (minimum 12 caractères selon l'ANSSI). Pour ajouter une "pass phrase", il faut ajouter **-des3** ou **-aes256** par exemple après 4096.

Vous pouvez visualiser la clef avec la commande :

```
cat /etc/apache2/ssl/ca.key
```

```
-----BEGIN PRIVATE KEY-----
MIIJQgIBADANBgkqhkiG9w0BAQEFAASCSSwwggkoAgEAAoICAQDNRCwo60hvTkaz
eG8MHX6JbZjqPJIX3bPDZNGVE71YEPDBAtaYGSxc4nPchzV5DkJgPWXX6wu7edV
NznfAwijekede0Lw4SXAphEqNiSgyOuR3L95HwLG55IFigkj2r5xC3y0Z6oHj6a
/NqjWprVS6eTgc2wJEJfuJuq3hatBjuGdk8rvQUi9teghJkVrU0X30yhbhtGu7Gpb
wFsApgKfgYdpzD8NDEzVpk9HWZrIuK3cgHuiX5No4AJps510zm+Ne1wQVvBFz5vI
4aoAMB7hh3qE5DIDKCIatUp0NfzUQnPhxnm1JJ5s2Yb/Jp8sQR45RQSVMe8/V
YoRDv281pJXDSug9yQdI1i2PFi5y/t0p6w0/c1NIEAXXWV70J6UWjUgNskyc6cr
919deo5MS4WCMp8UXKb2W+9h58m/zsU4uxA+Z9YAgMfp0Vp71X7n1trK3f00ffZk
6YXBYyH18miVvmSLHx0GMj/aVR0s/i0TYpjqSLD5xqX8SzcXc6lsH3Mp4mpf5I4d
yQDH1YPD7EQQ2Et0xR2qG1Qf4/dqCtL/48pzI87vwXt+bcUjGwR9cdK1Z0to2VMe
yzeb3+0K1R5ad3BSG/Vd+r1BjUm/IKAT86R0+BGK77Gc1AyQrPXbhqIodJEjC96
LsNa1fCivJcA7M2ZxL6F7MFU1sD81wIDAQABAoICAQEQDQxxg1zsYKW7tWYHIM8n
Evp4GUNzrFTRrIhW8wLXjF24+bcIvL0musiw4TZabvmXcgWaQyx/SZNXt7u3Xxm
Hz5S62Yv/EHFEcfJFZR9eP7C3dmpYy0XV2nobRL0Yoser0UAA8EII1YyIfVpJso1
P8QvUvKzBkudTfGTHpghWTZQDMp/ocpWCaigkHfmsT0ruMYtJ0wLNI4YjBCKg49
P19L1/0TnmHLqulu+4ViY3u8aMN6V1/U48xxAEhK8ji2JdEy43jBLMDU/1utvGPz
Zulv4SJG5YqekHpy2rRA5C2cK4YqZcfdaalXyfIx1ji5GuiPaD8HereVwNe66Xd
PE7977Bf9JUNqiBB3i8srpQmhDJyQg==
-----END PRIVATE KEY-----
```



CREATION DU CERTIFICAT AUTO-SIGNE DE L'AUTORITE DE CERTIFICATION

- Saisissez la commande suivante et complétez les options :

```
openssl req -new -x509 -days 365 -nodes -key ca.key > ca.crt
```

```
root@srvdeb:/etc/apache2/ssl# openssl req -new -x509 -days 365 -nodes -key ca.key > ca.crt
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:France
Locality Name (eg, city) []:Amiens
Organization Name (eg, company) [Internet Widgits Pty Ltd]:SaintRemi
Organizational Unit Name (eg, section) []:SISR
Common Name (e.g. server FQDN or YOUR name) []:srvdeb.sistr.tp
Email Address []:admin@sist.tp
```

Attention, renseignez
correctement le
Common Name qui
correspond à votre
FQDN !

A ce stade, le dossier **/etc/apache2/ssl** comporte 2 fichiers :

- le fichier **ca.key** correspond à la clé privée de notre Autorité de Certification
- le fichier **ca.crt** correspond au certificat auto-signé de notre Autorité de Certification

b) CREATION DU CERTIFICAT AUTO-SIGNE DU SERVEUR WEB APACHE

Dans cette partie, nous aurons 3 étapes : nous allons créer une clé privée (**.key**) pour notre serveur web Apache, un fichier de demande de signature (**.csr**) et un certificat auto-signé (**.crt**) pour notre serveur web Apache.

- ✓ la création de la clé privée du serveur web Apache
- ✓ la création du certificat auto-signé du serveur web Apache

CREATION DE LA CLE PRIVEE DU SERVEUR WEB APACHE

- Saisissez la commande :

```
openssl genrsa 4096 > clefpriveeapache.key
```

- Protégez la clé privée en changeant les droits :

```
chmod 400 clefpriveeapache.key
```

```
784652 drwxr-xr-x 2 root root 4096 24 sept. 16:09 .
784381 drwxr-xr-x 9 root root 4096 24 sept. 15:53 ..
784654 -rw-r--r-- 1 root root 2126 24 sept. 16:00 ca.crt
784653 -rw-r--r-- 1 root root 3272 24 sept. 15:54 ca.key
784655 -r----- 1 root root 3272 24 sept. 16:09 clefpriveeapache.key
```

CREATION DU FICHIER DE DEMANDE DE SIGNATURE A PARTIR DE LA CLE PRIVEE

- Saisissez la commande :

```
openssl req -new -key clefpriveeapache.key > demandesignature.csr
```

- Répondez aux questions pour générer le fichier **.csr** :

```

root@srvdeb:/etc/apache2/ssl# openssl req -new -key clefpriveeapache.key > demandesignature.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:FR
State or Province Name (full name) [Some-State]:France
Locality Name (eg, city) []:Amiens
Organization Name (eg, company) [Internet Widgits Pty Ltd]:SaintRemi
Organizational Unit Name (eg, section) []:SISR
Common Name (e.g. server FQDN or YOUR name) []:srvdeb.sisr.tp
Email Address []:admin@sisr.tp

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:l@boslr_

```

Ici vous devez saisir un mot de passe pour générer la demande (ne le perdez pas !)

A ce stade, votre dossier **/etc/apache2/ssl** contient 4 fichiers :

- le fichier **ca.crt** correspond au certificat de l'Autorité de Certification
- le fichier **ca.key** correspond à la clé privée de l'Autorité de Certification
- le fichier **cleprivapache.key** correspond à la clé privée de votre serveur Apache
- le fichier **demandesignature.csr** correspond à la demande à faire signer par l'Autorité de Certification

SIGNATURE DU FICHIER DE DEMANDE PAR L'AUTORITE DE CERTIFICATION

- Saisissez la commande suivante (attention, vérifiez bien la syntaxe) :

```
openssl x509 -req -in demandesignature.csr -out certifapache.crt -CA ca.crt -CAkey ca.key -CAcreateserial -days 365
```

```

root@srvdeb:/etc/apache2/ssl# openssl x509 -req -in demandesignature.csr -out certifapache.crt -CA ca.crt -CAkey ca.key -CAcreateserial -days 365
Certificate request self-signature ok
subject=C = FR, ST = France, L = Amiens, O = SaintRemi, OU = SISR, CN = srvdeb.sisr.tp, emailAddress = admin@sisr.tp

```

Remarque : L'option **-CAcreateserial** est à utiliser seulement la 1ère fois.

```

784652 drwxr-xr-x 2 root root 4096 24 sept. 16:24 .
784381 drwxr-xr-x 9 root root 4096 24 sept. 15:53 ..
784654 -rw-r--r-- 1 root root 2126 24 sept. 16:00 ca.crt
784653 -rw-r--r-- 1 root root 3272 24 sept. 15:54 ca.key
784658 -rw-r--r-- 1 root root 41 24 sept. 16:24 ca.srl
784657 -rw-r--r-- 1 root root 2004 24 sept. 16:24 certifapache.crt
784655 -r----- 1 root root 3272 24 sept. 16:09 clefpriveeapache.key
784656 -rw-r--r-- 1 root root 1809 24 sept. 16:18 demandesignature.csr

```

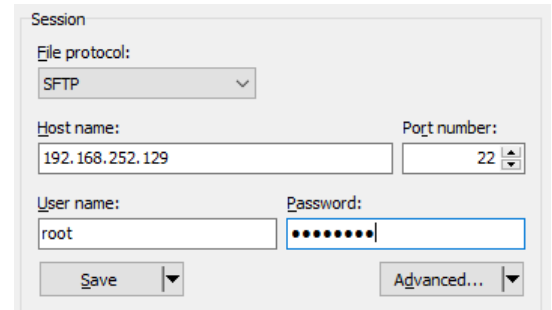
Le dossier « **/etc/apache2/ssl** » comporte maintenant 6 fichiers :

- le fichier **ca.crt** correspond au certificat de l'Autorité de Certification
- le fichier **ca.key** correspond à la clé privée de l'Autorité de Certification
- le fichier **ca.srl** contient un identifiant qui sera incrémenté pour une nouvelle demande de signature
- le fichier **certifapache.crt** correspond au certificat auto-signé de votre serveur Apache
- le fichier **cleprivapache.key** correspond à la clé privée de votre serveur Apache
- le fichier **demandesignature.csr** correspond à la demande à faire signer par l'Autorité de Certification

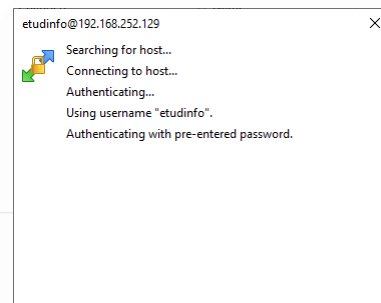
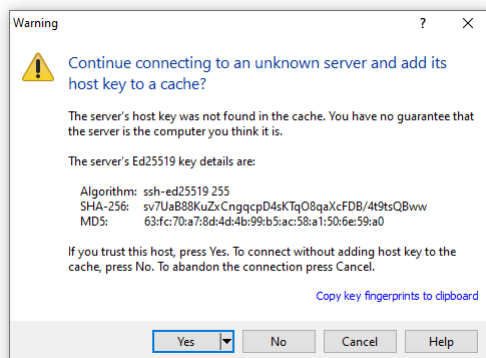
c) IMPORTATION DU CERTIFICAT DE L'AUTORITE DANS LE NAVIGATEUR WEB HOTE

Dans cette partie, nous allons ajouter notre Autorité de Certification dans le « magasin » de certificats de notre navigateur web. Pour réaliser ce travail, nous allons installer et utiliser WinSCP pour transférer le certificat vers notre machine Windows (mais il existe aussi d'autres méthodes).

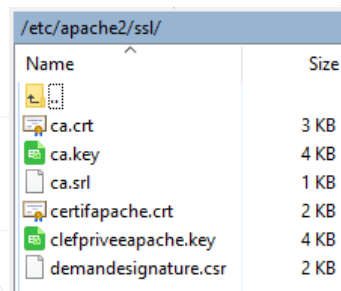
- Téléchargez et installez [WinSCP](#) depuis le lien
- Lancez WinSCP et connectez-vous à votre machine Debian



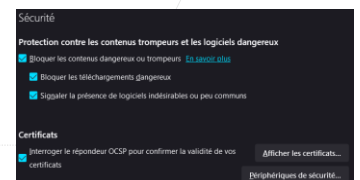
- WinSCP affiche une double fenêtre correspondant aux machines Windows et Debian :



- Allez dans le dossier **/etc/apache2/ssl** et faites glisser le fichier certificat de votre Autorité de Certificat **ca.crt** dans un dossier de votre machine Windows (afin de le récupérer) :



- Lancez votre navigateur (ici Firefox) et ouvrez les options puis **Paramètres**
- Dans **Vie privée et sécurité**, descendez jusqu'à la rubrique **Certificats**
- Cliquez le bouton **Afficher les certificats** :
- Cliquez le bouton **Importer** puis recherchez votre certificat **ca.crt** importé précédemment
- Cliquez l'option **Confirmer cette AC pour identifier les sites web**
- Cliquez le bouton **Ok** pour confirmer l'importation



En cliquant à nouveau le bouton **Afficher les certificats** on constate que le certificat de notre Autorité est bien

présent dans le **magasin des autorités racines de confiance** :

Nom du certificat	Périphérique de sécurité	
▼ SaintRemi		
srvdeb.sisr.tp	Sécurité personnelle	

Maintenant que le certificat de l'Autorité est importé dans le navigateur, il faut modifier la configuration du fichier **/etc/apache2/sites-available/default-ssl.conf** pour stipuler l'emplacement des fichiers certificats du serveur Apache, fichiers qui ont été générés précédemment dans **/etc/apache2/ssl**.

- Ouvrez le fichier **/etc/apache2/site-available/default-ssl.conf**

```
<VirtualHost *:443>
    ServerAdmin webmaster@localhost
    ServerName srvdeb.sisr.tp
    DocumentRoot /var/www/html

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf

    # SSL Engine Switch:
    # Enable/Disable SSL for this virtual host.
    SSLEngine on

    # A self-signed (snakeoil) certificate can be created by installing
    # the ssl-cert package. See
    # /usr/share/doc/apache2/README.Debian.gz for more info.
    # If both key and certificate are stored in the same file, only the
    # SSLCertificateFile directive is needed.
    SSLCertificateFile /etc/apache2/ssl/certifapache.crt
    SSLCertificateKeyFile /etc/apache2/ssl/clefpriveeapache.key
```

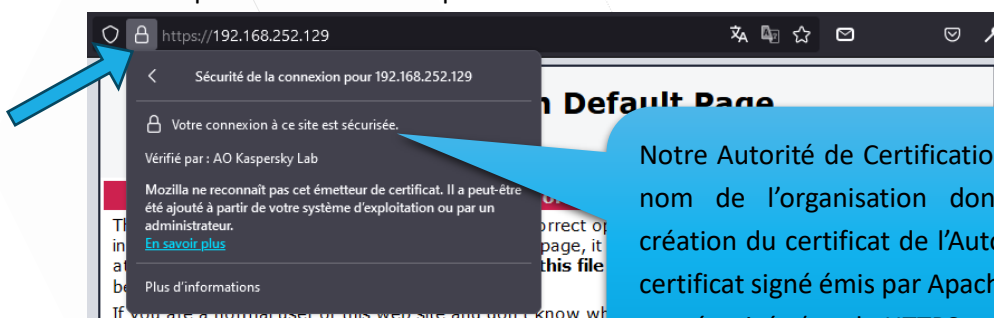
Ici, indiquer l'emplacement des fichiers **.crt** et **.key** du serveur Apache

- Relancez Apache en saisissant

```
systemctl restart apache2
```

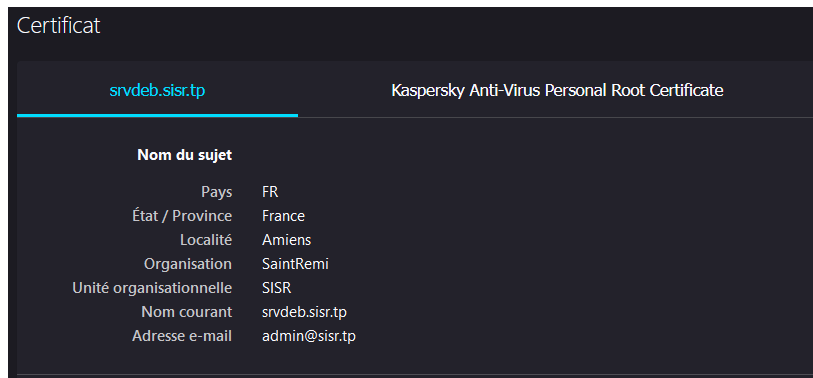
- Connectez-vous à **https://srvdeb.sisr.tp** : il n'y a plus d'alerte de sécurité !

Votre site web HTTPS est pleinement reconnu par votre Autorité :



Notre Autorité de Certification (SaintRemi du nom de l'organisation donné lors de la création du certificat de l'Autorité) a validé le certificat signé émis par Apache : la connexion est sécurisée (mode HTTPS activé).

En cliquant le cadenas, on constate que la connexion est devenue pleinement sécurisée :



6. REDIRECTION AUTOMATIQUE DES REQUETES HTTP VERS HTTPS

Dans ce guide, nous avons activé le site https par défaut d'Apache mais nous avons laissé le site HTTP actif également. Il n'est pas utile de désactiver le site par défaut du fichier **000-default.conf**.

Il faut plutôt configurer une redirection permanente des requêtes HTTP qui arrivent sur le port 80 vers le site sécurité HTTPS (port 443), en modifiant simplement le fichier **000-default.conf** :

- Editez le fichier **/etc/apache2/sites-available/000-default.conf**
- Ajoutez la ligne **Redirect permanent / https://srvdeb.sisr.tp/ :**

```
ServerAdmin webmaster@localhost
ServerName srvdeb.sisr.tp
Redirect permanent / https://srvdeb.sisr.tp/
DocumentRoot /var/www/html
```

- Relancez Apache en saisissant la commande :

```
systemctl restart apache2
```

- Saisissez, dans votre navigateur : **srvdeb.sisr.tp**
vous êtes redirigé(e) vers le site HTTPS automatiquement !

ANALYSE DES LOGS SUR SERVEUR APACHE

```
ls -ila /var/log/apache2/
```

```
total 24
1046480 drwxr-x--- 2 root adm 4096 24 sept. 15:00 .
1046451 drwxr-xr-x 8 root root 4096 24 sept. 15:00 ..
1046469 -rw-r----- 1 root adm 3285 24 sept. 18:36 access.log
1046688 -rw-r----- 1 root adm 1462 20 sept. 19:01 access.log.1
1046490 -rw-r----- 1 root adm 3856 24 sept. 18:36 error.log
1046687 -rw-r----- 1 root adm 1413 20 sept. 22:30 error.log.1
1046689 -rw-r----- 1 root adm 0 19 sept. 18:41 other_vhosts_access.log
```

QUELQUES COMMANDES UTILES

Activer le mode SSL	a2enmod ssl
Activer un site Apache	a2ensite xxx.conf (le fichier est dans /etc/apache2/sites-available)
Désactiver un site Apache	a2dissite xxx.conf (le fichier est dans /etc/apache2/sites-available)
Vérifier les sites Apache actifs	Se rendre dans /etc/apache2/sites-enabled

7. PERSONNALISATION D'APACHE

Nous allons travailler avec les modules Apache situés dans les répertoires **/etc/apache2/** :

- › **mods-available** contient la liste des modules d'apache installés
- › **mods-enabled** contient la liste des modules utilisés
- Afficher le contenu du fichier **/mods-available/dir.conf** et **/mods-enabled/dir.conf** :

```
cat /etc/apache2/mods-available/dir.conf
cat /etc/apache2/mods-enabled/dir.conf
```

```
root@srvdeb:/etc/apache2# cat /etc/apache2/mods-available/dir.conf
DirectoryIndex index.html index.cgi index.pl index.php index.xhtml index.htm
root@srvdeb:/etc/apache2# cat /etc/apache2/mods-enabled/dir.conf
DirectoryIndex index.html index.cgi index.pl index.php index.xhtml index.htm
root@srvdeb:/etc/apache2# _
```

- Modifier le fichier **/mods-enabled/dir.conf** en ajoutant le fichier par défaut **default.html** dans la liste de la directive **DirectoryIndex**.
- Dans le navigateur, ouvrez la page <http://srvdeb.sisr.tp/default.html>
- Créer une page **default.html** dans le répertoire **/var/www** puis tester dans le navigateur
- Renommer la page **index.html** en **index.html.sav**

```
mv index.html index.html.sav
```

- Dans le navigateur, ouvrez la page <http://srvdeb.sisr.tp>

Pour le moment, le répertoire par défaut de votre site est situé dans **/var/www**

a) Personnaliser le service

Nous allons désormais créer un répertoire spécifique pour chaque utilisateur. Pour rappel, vous pouvez créer un utilisateur nommé **etudsisr** avec la commande :

```
useradd -m -d /home/etudsisr/ -s /bin/bash etudsisr
```

puis définir son mot de passe avec :

```
passwd etudsisr
```

```
root@srvdeb:/var/www/html# useradd -m -d /home/alice/ -s /bin/bash alice
root@srvdeb:/var/www/html# passwd alice
Nouveau mot de passe :
Retapez le nouveau mot de passe :
passwd : mot de passe mis à jour avec succès
root@srvdeb:/var/www/html#
```

- Ajouter un utilisateur portant votre prénom et un second avec le prénom de votre voisin.



L'objectif ici, est de pouvoir accéder au site des différents utilisateurs, dans leur espace privé du répertoire **homedir**, via l'url <https://srvdeb.sisr.tp/~monnom/>

Dans leur espace privé, chaque utilisateur doit disposer d'un répertoire **public_html** :

- Activer le module **userdir** :

```
a2enmod
userdir
systemctl restart apache2
```

- Créer un répertoire **public_html** dans le répertoire **homedir** de chaque utilisateur avec une page **index.html** différente pour chaque utilisateur.
- Tester l'url <https://srvdeb.sisr.tp/~monnom/>

8. CONFIGURATION AVANCÉE D'APACHE

a) Accès par authentification

Les fichiers **.htaccess** sont des fichiers de configuration d'Apache, permettant de définir des règles dans un répertoire et dans tous ses sous-répertoires (qui n'ont pas de tel fichier à l'intérieur). On peut les utiliser pour protéger un répertoire par mot de passe, ou pour changer le nom ou l'extension de la page index, ou encore pour interdire l'accès au répertoire.

Les fichiers **.htaccess** peuvent être utilisés dans n'importe quel répertoire virtuel ou sous-répertoire. Les principales raisons d'utilisation des fichiers **.htaccess** sont :

- Gérer l'accès à certains fichiers.
- Protéger l'accès à un fichier par un mot de passe 
- Ajouter un mime-type
- Définir des pages d'erreurs personnalisées
- Protéger l'accès à un répertoire par un mot de passe
- Principe des fichiers .htaccess

Le fichier **.htaccess** est placé dans le répertoire dans lequel il doit agir. Il agit ainsi sur les permissions du répertoire qui le contient et de tous ses sous-répertoires. Vous pouvez placer un autre fichier **.htaccess** dans un sous-répertoire d'un répertoire déjà contrôlé par un fichier **.htaccess**. Le fichier **.htaccess** du répertoire parent reste en "activité" tant que les fonctionnalités n'ont pas été réécrites. Pensez à bien positionner la directive **AllowOverride** pour que vos fichiers **.htaccess** soient pris en compte.



Modifier le fichier **default** dans **/etc/apache2/sites-available** pour passer **AllowOverride** de la valeur **None** à **All**.

b) Empêcher l'accès à des ressources

Un fichier .htaccess est composé de deux sections :

- 1 {
 - AuthUserFile** /repertoire/de/votre/fichier/.FichierDeMotDePasse
 - AuthGroupFile** /repertoire/de/votre/fichier/.FichierDeGroupe
 - AuthName** "Accès protégé"
 - AuthType** Basic
- 2 {
 - <Limit GET POST>
 - Require valid-user** {instruction d'accès à satisfaire}
 - </Limit>

1) Une première section contient les chemins vers les fichiers contenant les définitions de groupes et d'utilisateurs :

- **AuthUserFile** définit le chemin d'accès absolu vers le fichier de mot de passe.
- **AuthGroupFile** définit le chemin d'accès absolu vers le fichier de groupe.
- **AuthName** entraîne l'affichage dans le navigateur Internet de : Tapez votre nom d'utilisateur et votre mot de passe. Domaine : "Accès protégé"
- **AuthType Basic** précise qu'il faut utiliser AuthUserFile pour l'authentification

2) Une seconde section contient la définition des conditions d'accès :

- La balise **LIMIT** possède en attribut la valeur **GET** (en majuscule) et/ou la valeur **POST**, afin de définir le type de méthode du protocole **HTTP** auxquelles la restriction s'applique
- **Require valid-user** précise que l'on autorise uniquement les personnes identifiées. Il est également possible de préciser explicitement le nom des personnes autorisées à s'identifier : **require user {username}**

Le chemin d'accès vers les fichiers de mots de passe et de groupes est de la forme suivante :
/repertoire1/repertoire2/.../.FichierDeMotDePasse

c) Protéger un répertoire par un mot de passe

Il s'agit d'une des applications les plus utiles du fichier **.htaccess** car elle permet de définir de façon sûre (à l'aide d'un login et d'un mot de passe) les droits d'accès à des fichiers par certains utilisateurs. La syntaxe est la suivante :

```
AuthUserFile {emplacement du fichier de mot de passe}
AuthGroupFile {emplacement du fichier de groupe}
AuthName "Accès protégé"
AuthType Basic
<LIMIT GET POST>
Require valid-user
</LIMIT>
```



- La directive **AuthUserFile** permet de définir l'emplacement du fichier contenant les logins et les mots de passe des utilisateurs autorisés à accéder à une ressource donnée.
- La directive **AuthGroupFile** permet de définir l'emplacement du fichier contenant les groupes d'utilisateurs autorisés à s'identifier. Il est possible d'outrepasser cette déclaration en déclarant le fichier suivant : **/dev/null**.
- Créez un fichier **.htaccess** qui demande de s'identifier pour accéder au répertoire secret que vous avez créé.
- Utilisez un fichier de mot de passe contenant au moins 4 noms et vérifiez que seulement ceux-ci fonctionnent.

Le fichier de mot de passe est un fichier texte devant contenir sur chacune de ses lignes le nom de chaque utilisateur suivi des deux points (:), puis du mot de passe crypté (solution recommandée) ou en clair. Ce fichier de mot de passe ne devrait pas être mis dans un répertoire virtuel Internet.

Remarque : Il ne faut pas utiliser de fichier de mots de passe non chiffrés, en effet les versions non chiffrées ne sont plus supportées par défaut par les navigateurs actuels

d) Crypter les mots de passe

Apache fournit un outil permettant de générer facilement des mots de passe cryptés (aussi bien sous Windows que sous Unix), il s'agit de l'utilitaire **htpasswd** accessible dans le sous-répertoire **bin** d'Apache. La syntaxe de cet utilitaire est la suivante :

- Pour créer un nouveau fichier de mots de passe :
htpasswd -c {chemin du fichier de mot de passe} utilisateur
- Pour ajouter un nouvel utilisateur/mot de passe à un fichier existant :
htpasswd {chemin du fichier de mot de passe} utilisateur
- Créez un fichier de mot de passe pour tester l'accès restreint à certains répertoires.

e) Empêcher l'accès à un répertoire par un domaine

La syntaxe pour bloquer l'accès d'un répertoire par un domaine est la suivante :

Order (allow,deny ou deny,allow)
Allow (all, [liste de domaine])
Deny (all, [liste de domaine])

Exemple :

Order deny,allow



Deny from .domaine.com

Toutes les requêtes provenant du domaine **.domaine.com** ne pourront avoir accès aux ressources comprises dans le répertoire et ses sous-répertoires. La commande **Order** sert à préciser explicitement que la commande **Deny** va bien annuler l'effet de **Allow** et non l'inverse

- Réaliser un accès qui ne sera possible que pour les utilisateurs "thomas" et "gaetan" à partir de l'adresse IP de votre choix et avec le bon mot de passe.

f) Empêcher l'accès à fichier particulier

Par défaut, Apache applique les restrictions du fichier **.htaccess** à l'ensemble des fichiers du répertoire dans lequel il se trouve ainsi qu'à tous les fichiers contenus dans ses sous-répertoires. Il est également possible de restreindre l'accès pour un ou plusieurs fichiers du répertoire grâce à la balise **<Files>**.

Exemple de restriction au fichier secret.html :

```
<Files secret.html>
AuthUserFile /var/conf/www/.pass
AuthGroupFile /dev/null
AuthName "Accès sécurisé à mon site"
AuthType Basic
<LIMIT GET POST>
require user gaetan
</LIMIT>
</Files>
```

- Créez un fichier **.htaccess** qui demande de s'identifier pour accéder à l'url **http://IpMonServeur/secret/info.htm** et qui limite cet accès à l'utilisateur **thomas**.

Remarque : Il ne faut utiliser qu'une seule balise **<Files>** par fichier. Sinon, l'erreur suivante est reportée dans le fichier de log des erreurs : **.htaccess: Multiple arguments not (yet) supported.**

g) Protéger un répertoire par un login

Cette méthode permet une authentification de bas niveau uniquement par le nom de l'utilisateur. La syntaxe est la suivante :

Require (user [liste des utilisateurs], group [liste des groupes], valid-user)

Voici un exemple de ligne du fichier **.htaccess** :

Require user gaetan thomas



Tout utilisateur souhaitant rentrer dans le répertoire ou un de ses sous-répertoires sera refusé sauf s'il s'identifie en donnant un nom figurant dans la liste.

- Créez un fichier **.htaccess** qui protège le répertoire secret2 pour l'utilisateur **paul**.

h) Ajouter un Mime-Type à un répertoire

Un type MIME (en anglais MIME type) est un ensemble de types de fichiers standard, permettant d'associer une extension de fichier donnée à une application, afin d'automatiser le lancement de l'application.

La syntaxe est la suivante :

AddType (mime/type [liste d'extension])

Voici un exemple de mise en oeuvre du fichier .htaccess :

AddType image/x-wordpad RFT

AddType application/x-httpd-php .php3

AddType application/x-httpd-php .htm

Le serveur enverra au navigateur Internet le fichier en lui disant de lancer le programme WordPad et de lui donner le fichier. Habituellement, ceci est utilisé pour des fichiers nécessitant un Plug-in particulier non reconnu par votre navigateur.

Cette commande permet aussi d'annuler tout élément prédéfini. Ainsi, vous pouvez enregistrer un fichier .wav avec une extension .gif et préciser au navigateur de considérer les fichiers .gif comme des fichiers audio ! En pratique, on pourra donc utiliser cette commande pour ordonner à l'interpréteur PHP d'évaluer d'autres extensions de fichier, .htm par exemple.

i) Forcer tous les fichiers d'un répertoire à un Mime-Type

Voici la syntaxe à adopter : **ForceType (mime/type)**

Par exemple avec la ligne suivante, tous les fichiers du répertoire contenant le fichier **.htaccess** seront considérés comme étant des fichiers .jpg quelle que soit leur extension : **ForceType image/jpg**

- Permettre à un répertoire nommé **imgpng** de n'accepter que les fichiers de type "**png**".
- Tester avec une page php de upload vers le répertoire imgpng.

j) Définir les extensions de fichiers par défaut

La syntaxe à suivre est : **DefaultType (mime/type)**

Par exemple : **DefaultType text/html**

Cette option vous permet de définir le comportement par défaut du navigateur face à des extensions lui étant inconnues. Ici, il prendra tout fichier inconnu en tant que document HTML.



- Créez un fichier **.htaccess** pour un répertoire nommé **uploaddocs** qui n'accepte que les fichiers de type : txt, pdf, doc, docx, xls et xlsx.
- Tester avec une page d'upload vers ce répertoire.

k) Personnalisation des messages d'erreurs

Il s'agit d'une fonctionnalité pratique car elle permet de définir une page par défaut pour un type d'erreur donné. Cela permet d'une part de guider l'utilisateur au lieu d'afficher la banale page d'erreur du navigateur, ainsi que d'égayer la navigation même en cas d'erreur.

ErrorDocument (code-à-3-chiffres [nom du fichier ou texte ou url])

Les deux lignes suivantes permettent de définir des pages d'erreurs personnalisées au cas où l'accès à un document serait interdit ou bien que le document n'existe pas :

ErrorDocument 403 /erreurs/403.php3

ErrorDocument 404 /erreurs/404.php3

Ceci vous permet de donner un message d'erreur personnalisé remplaçant les fichiers fournis avec le navigateur. Voici quelques-unes des erreurs les plus courantes à personnaliser :

401 Unauthorized : la personne n'a pas passé avec succès l'identification.

403 Forbidden : le serveur n'a pas le droit de répondre à votre requête.

404 Not Found : le serveur n'a pas trouvé le document souhaité.

- Créez un fichier **.htaccess** afin de personnaliser l'erreur **Not Found** de votre site web.
- Chercher à proposer à vos utilisateurs la création d'un répertoire **repererror** dans leur **public_html** afin de leur offrir la possibilité de personnaliser leurs propres pages d'erreurs.

l) Changer le fichier index par défaut

Le fichier index.html est le fichier affiché lorsqu'il n'y a pas de nom de fichier défini dans l'URL. On a déjà vu que la syntaxe pour effectuer ce type d'opération est la suivante : **DirectoryIndex (fichiers)**

Voici un exemple de mise en application :

DirectoryIndex index.php index.html index.phtml /erreurs/403.php

Lorsque vous essayez d'accéder au répertoire sans préciser la page à afficher, Apache va avoir recours à la directive **DirectoryIndex**. En général, cette directive pointe par défaut vers index.html puis index.htm. Dans l'exemple ci-dessus, Apache va commencer par chercher index.php, puis index.html, et ensuite index.phtml. Si aucun de ces fichiers existent, la page 403.php sera affichée pour éviter de lister le répertoire.

- Pour le répertoire **repxhtml**, donner la priorité aux fichiers dont l'extension est **"xhtml"**.

